SILVER OAK UNIVERSITY EDUCATION TO INNOVATION	Course Code: 3040243338 Course Name: Mobile Game Development	
	SEMESTER:	5

Unit - 3

Advanced Game Development Concepts

What is AI in Gaming?

Artificial intelligence by having gaming means making use of artificial intelligence practices to generate more dynamic, adaptive and gripping video game experiences. It is as if it fills the void that exists the virtual environment to make interactions and gameplay more natural.

NPC Behavior using State Machines

1. Introduction

In video games, **Non-Player Characters (NPCs)** are characters controlled by the computer instead of the player. To make NPCs behave intelligently and realistically, developers use **Finite State Machines (FSMs)**.
A **State Machine** is a way of organizing NPC behavior into different states (such as **Idle, Patrol, Chase, Attack**) and switching between them based on conditions.

For example:

- An enemy NPC can **patrol** when no player is nearby.
- If it detects the player, it **chases** them.
- If close enough, it **attacks**.
- If the player escapes, the NPC **returns to patrol**.

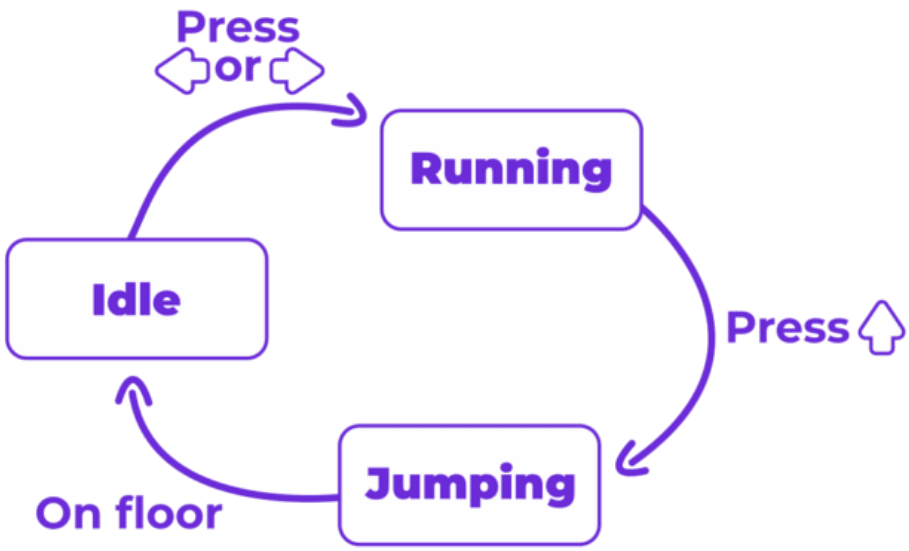
This structure makes NPC behavior more predictable, manageable, and realistic.

2. Uses of State Machines in Games

- **Organized NPC logic** → Easier to design and debug.
- **Realistic gameplay** → NPCs react differently depending on situations.
- **Reusability** → Same state machine logic can be applied to multiple NPCs.
- **Flexibility** → Easy to expand (add new states like "Search" or "Flee").
- **Performance friendly** → Runs efficiently compared to complex AI algorithms.

3. Common NPC States

- **Idle** → NPC stands still, waits, or plays idle animation.
- **Patrol** → NPC moves between waypoints or predefined paths.
- **Chase** → NPC follows the player when detected.
- **Attack** → NPC attacks if close to the player.
- **Return/Reset** → NPC goes back to patrol if the player escapes.



Steps to Implement NPC State Machine in Godot Engine

Step 1: Create NPC Scene

1. Open Godot and create a **KinematicBody2D** (for NPC movement).
2. Rename it to Enemy.
3. Add child nodes:
    - **Sprite** (for enemy graphics).
    - **CollisionShape2D** (for detecting collisions).
    - **Timer** (optional, for attack cooldowns).

Step 2: Define States in Script

Attach a new script to the Enemy node (enemy.gd).
Define constants for different states:

```
extends KinematicBody2D


enum State { IDLE, PATROL, CHASE, ATTACK }
```

```
var current_state = State.IDLE
var speed = 100
var player = null
```

Step 3: Implement State Behaviors

Inside _physics_process(delta), check the current state and run logic:

```
func _physics_process(delta):
    match current_state:
        State.IDLE:
            idle_behavior()
        State.PATROL:
            patrol_behavior()
        State.CHASE:
            chase_behavior()
        State.ATTACK:
            attack_behavior()
```

Step 4: Write Behavior Functions

Example behaviors for each state:

```
func idle_behavior():
    # Enemy does nothing or plays idle animation
    velocity = Vector2.ZERO

func patrol_behavior():
    # Move between waypoints
    velocity = Vector2.RIGHT * speed
    move_and_slide(velocity)

func chase_behavior():
    if player:
        var direction = (player.global_position - global_position).normalized()
```

```
        velocity = direction * speed
        move_and_slide(velocity)

func attack_behavior():
    # Perform attack (example: reduce player health)
    print("Enemy attacks!")
```

Step 5: Add State Transitions

Use conditions to change states:

```
func _process(delta):
    if current_state == State.IDLE and player_in_range():
        current_state = State.CHASE
    elif current_state == State.CHASE and player_close():
        current_state = State.ATTACK
    elif current_state == State.ATTACK and not player_close():
        current_state = State.CHASE
    elif current_state == State.CHASE and not player_in_range():
        current_state = State.PATROL
```

Helper functions for detection:

```
func player_in_range() -> bool:
    return player and global_position.distance_to(player.global_position) < 200

func player_close() -> bool:
    return player and global_position.distance_to(player.global_position) < 50
```

Step 6: Connect Player

- In the main scene, instance the Enemy.
- Assign the player variable by referencing the Player node:

```
func _ready():
    player = get_node("/root/MainScene/Player")
```

5. Example Flow

1. NPC starts in **Idle**.
2. If player is within 200px → switch to **Chase**.
3. If player is very close (50px) → switch to **Attack**.
4. If player escapes → return to **Patrol**.

2D Path Finding in Godot

Path finding is a function that determines the shortest possible path from an object to its destination, for example, when moving an object to a certain destination.

Up to Godot 3.4, the **Navigation** node was used to implement path finding. This was not particularly inconvenient, but the methodology for game development using it was limited and inapplicable in some areas. This time, I would like to introduce an implementation method using **Navigation Server**, which was added to Godot 3.5. This is a backport from Godot 4, which is currently under active development. This article is intended for users of Godot 3.5 or later. Users of Godot version 3.4 or earlier should take note.

Creating a new project

Start Godot Engine and create a new project. You can name your project as you like. If you can't think of one, let's call it "2D Path Finding Start".

Updating project settings

Once the editor appears, we should update project settings for the entire project.

First, set the display size for the game.

1. Open the "Project" menu > "Project Settings
   Select "Display" > "Window" from the sidebar of the "General" tab.
2. In the "Size" section, change the values of the following items.
   - Width: 512
   - Height: 320
   - Test Width: 768
   - Test Height: 480
3. In the "Stretch" section, change the values of the following items

- Mode: 2d
- Aspect: keep

4. Switch to the “Input Map” tab and add “move_to” to the action.
5. Assign the “left mouse button” to the “move_to” action.

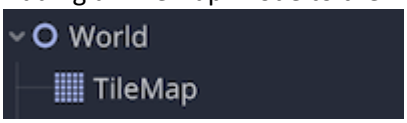
Creating a new World scene

The first step is to create a “World” scene to prepare the game world. 1. Select “Scene” menu > “New Scene”.

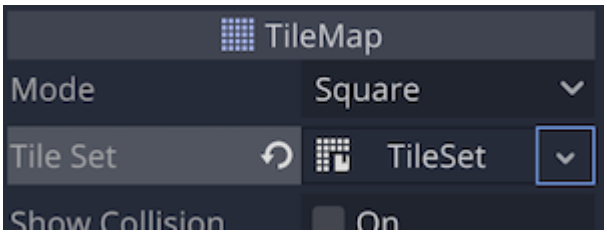
1. Select “Other Node” in “Generate Root Node”.
2. When the root node of the “Node2D” class is generated, rename it to “World”.
3. Save the scene. Create a folder and save the scene with the file path “res://Scene/World.tscn”.

Adding and editing a TileMap node

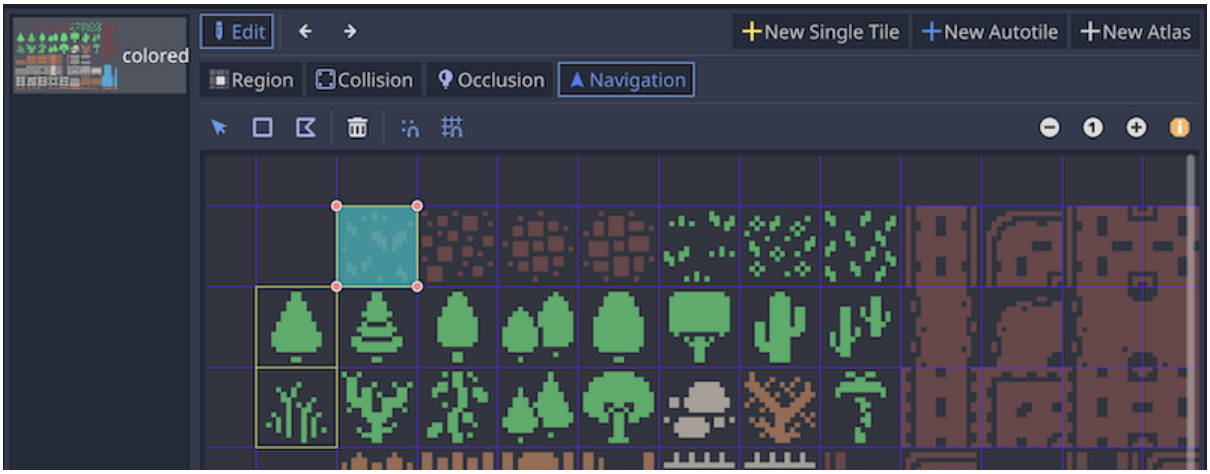
1. Adding a “TileMap” node to the “World” root node.



2. In the inspector, apply the “New TileSet” resource to the “TileSet” property of the “TileMap” node.

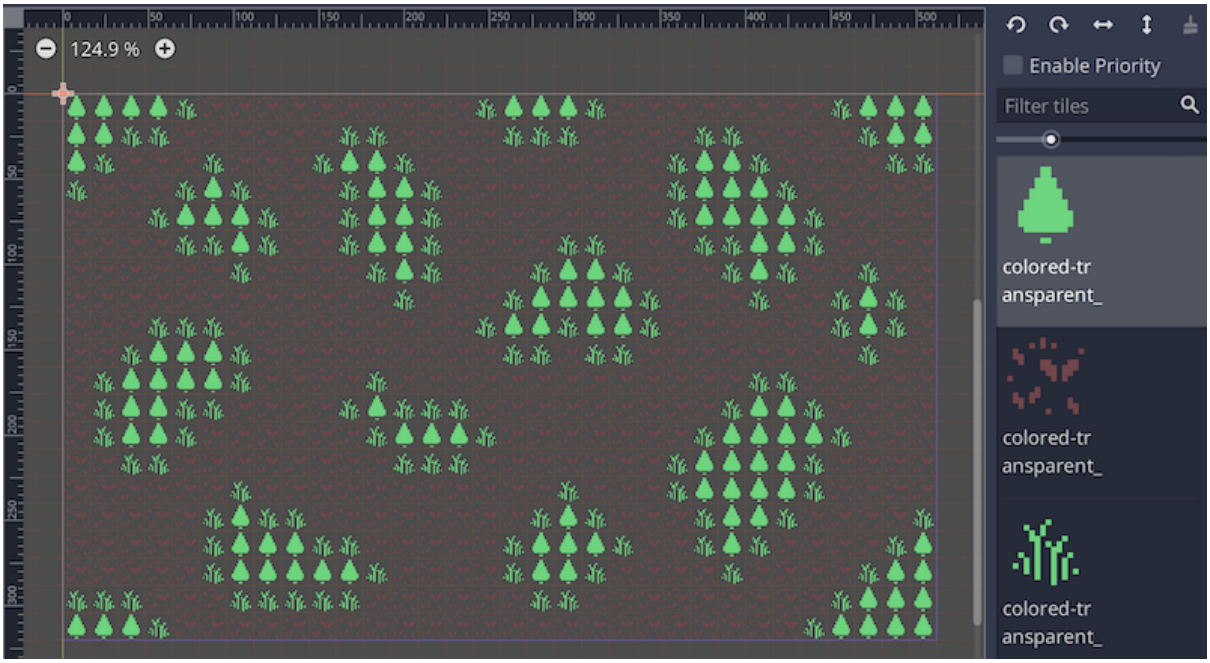


3. Click on the applied “TileSet” resource to open the TileSet panel at the bottom of the editor.
4. Add imported KENNEY “res://colored-transparent_packed.png” resource file by dragging it to the left sidebar of the TileSet panel.
5. Select the added texture sheet and prepare the following three single tiles.

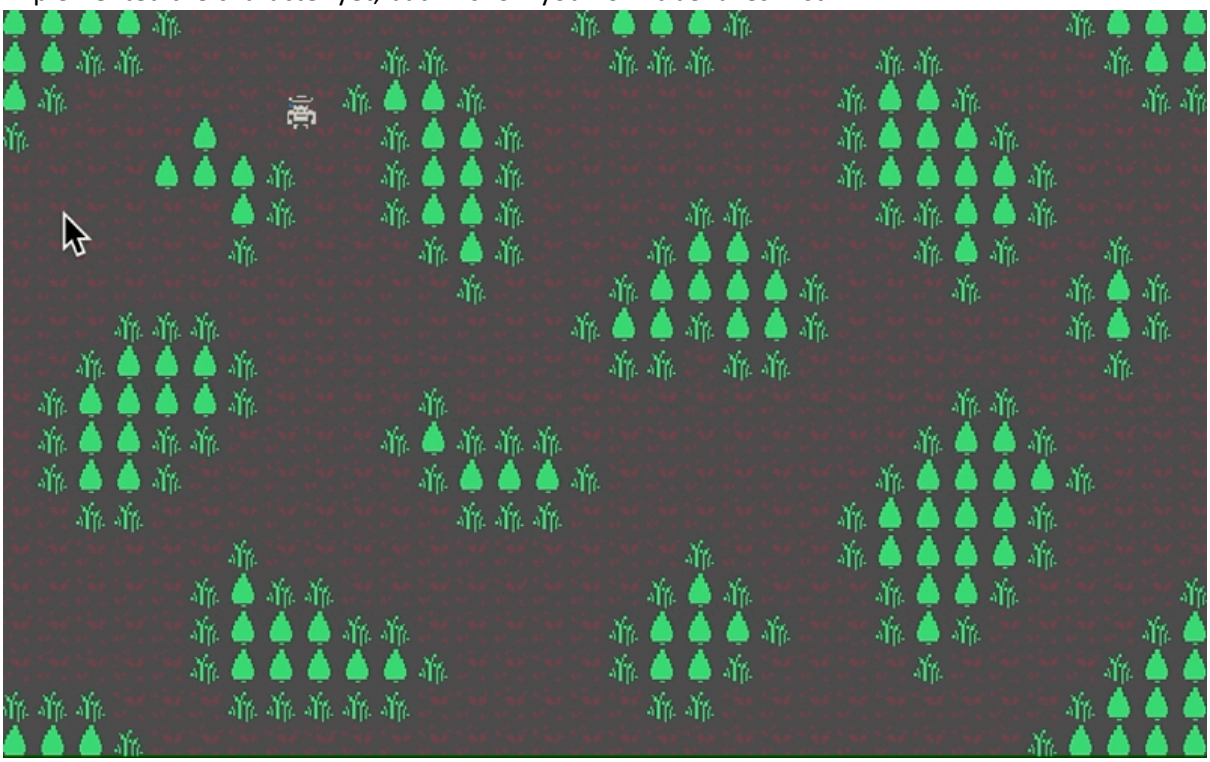


- Tile for the character's pathway.
  *Tiles with this navigation area set are the target of the pathfinding.
  - Use gravel tiles
  - Collision polygon: not required
  - Navigation polygon: **Needed**
- Tiles that characters do not pass through but do not collide with each other.
  *Use as a margin so that the character does not get caught by tree tiles with collision geometry when moving along the path.
  - Use grass texture
  - Collision polygons: not needed
  - Navigation polygons: not needed
- Tiles that characters cannot pass through and have collision detection.
  *Use as impassable obstacles during path finding.
  - Use tree texture
  - Collision polygon: **Needed**
  - Navigation polygons: not required

6. Select "TileMap" in the scene dock and create a tile map on the 2D workspace. Below is a sample. It is OK if the gravel tiles provide some pathways (navigation areas).



It is important to note, however, that once you place the tree tiles for obstacles, be sure to place the grass tiles for margins around them. Otherwise, the character will try to pass right through the wooden tiles when moving along the path, and will get stuck and not be able to move. This is something I would like to see improved in future Godot updates. I haven't implemented the character yet, but I'll show you how it behaves first.



7.2D Navigation in Godot 4

2D navigation overview

Godot provides multiple objects, classes and servers to facilitate grid-based or mesh-based navigation and pathfinding for 2D and 3D games. The following section provides a quick overview over all available navigation related objects in Godot for 2D scenes and their primary use.

Godot provides the following objects and classes for 2D navigation:

- **NavigationRegion2D Node**

A Node that holds a NavigationPolygon resource that defines a navigation mesh for the NavigationServer2D.

  - The region can be enabled / disabled.
  - The use in pathfinding can be further restricted through the **navigation_layers** bitmask.
  - The NavigationServer2D will join the navigation meshes of regions by proximity for a combined navigation mesh.

- **NavigationLink2D Node**

A Node that connects two positions on navigation meshes over arbitrary distances for pathfinding.

  - The link can be enabled / disabled.
  - The link can be made one-way or bidirectional.
  - The use in pathfinding can be further restricted through the **navigation_layers** bitmask.

Links tell the pathfinding that a connection exists and at what cost. The actual agent handling and movement needs to happen in custom scripts.

- **NavigationAgent2D Node**

A helper Node used to facilitate common NavigationServer2D API calls for pathfinding and avoidance. Use this Node with a Node2D inheriting parent Node.

- **NavigationObstacle2D Node**

A Node that can be used to affect and constrain the avoidance velocity of avoidance enabled agents. This Node does NOT affect the pathfinding of agents. You need to change the navigation meshes for that instead.

The 2D navigation meshes are defined with the following resources:

- **NavigationPolygon Resource**

A resource that holds 2D navigation mesh data. It provides polygon drawing tools to allow defining navigation areas inside the Editor as well as at runtime.

  - The NavigationRegion2D Node uses this resource to define its navigation area.
  - The NavigationServer2D uses this resource to update the navigation mesh of individual regions.

- The TileSet Editor creates and uses this resource internally when defining tile navigation areas.

Setup for 2D scene

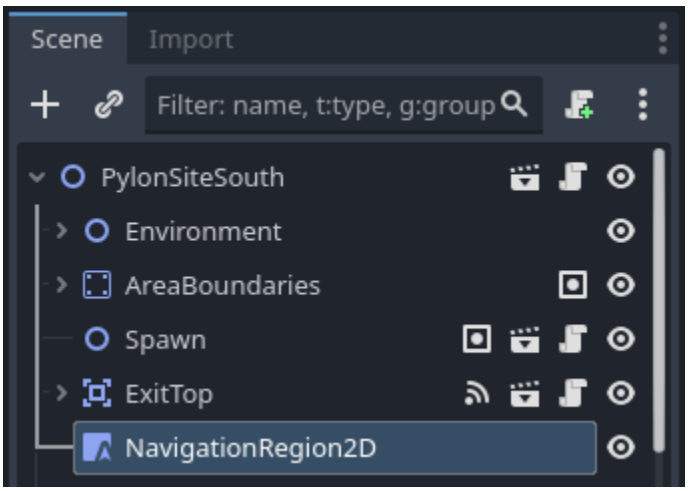
The following steps show the basic setup for minimal viable navigation in 2D. It uses the NavigationServer2D and a NavigationAgent2D for path movement.

Ingredients

- *NavigationRegion2D*
- *NavigationAgent2D* (one per node that should navigate)
- A little bit of scripting

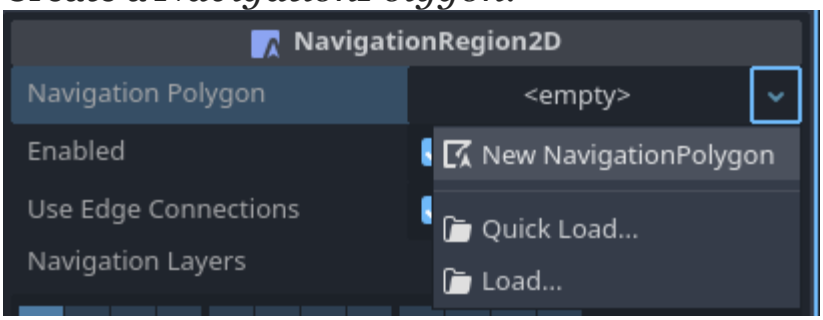
Steps

1. Add a NavigationRegion2D Node to the scene.
2. Click on the region node and add a new NavigationPolygon Resource to the region node.



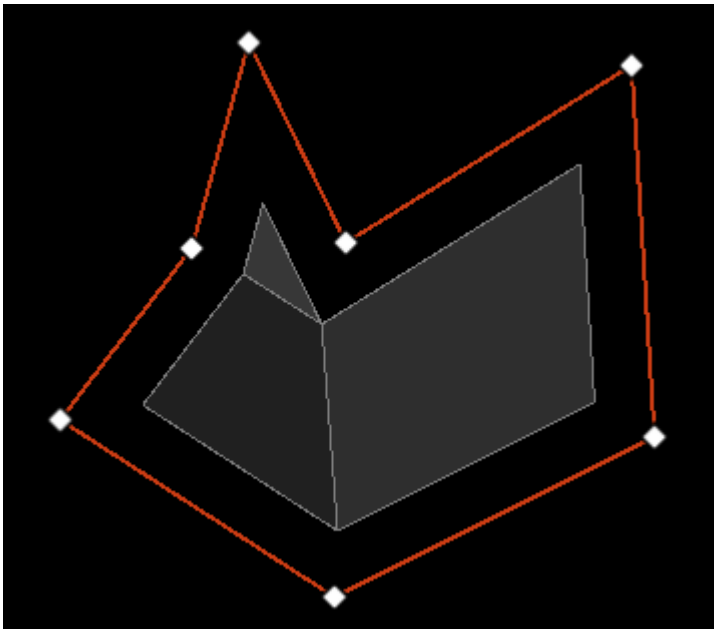
NavigationRegion2D in the scene tree

3. Create a *NavigationPolygon*.



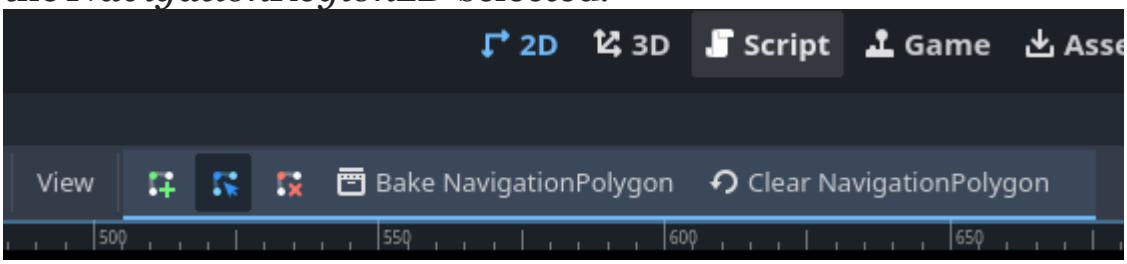
New NavigationPolygon in the inspector

4. Draw a region by clicking in the editor to place points.



An auto-baked region drawn in the editor

5. The region should bake automatically, but you can force it to re-bake by clicking the “Bake NavigationPolygon” button, which appears in the top toolbar when you have the *NavigationRegion2D* selected.



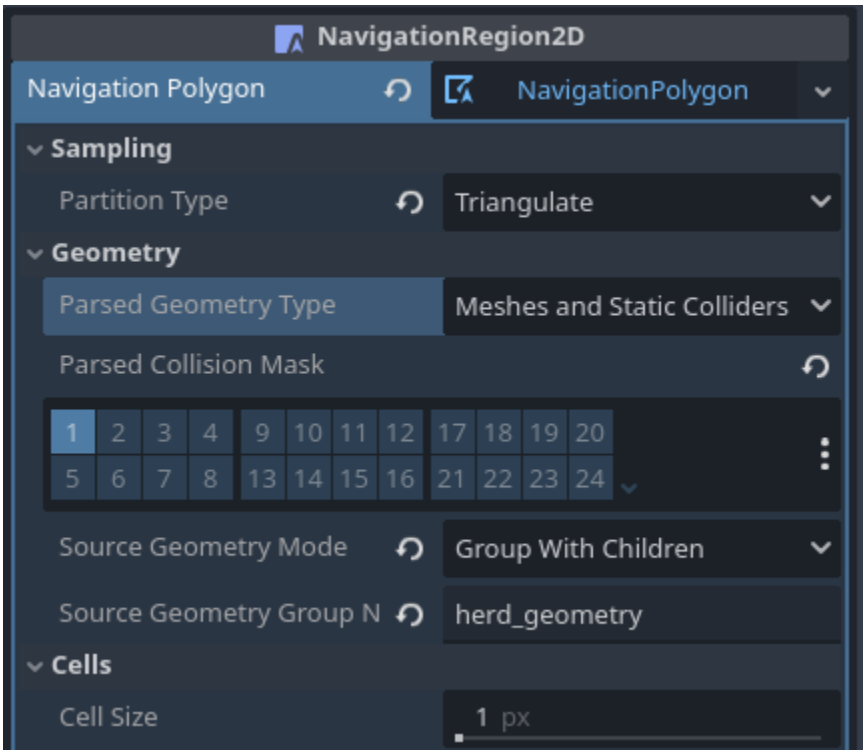
The elusive “Bake NavigationPolygon” button

6. I’ve used the following *NavigationRegion2D* settings effectively:
7. Create your *NavigationAgent2D* nodes. I made these children of the *Character2D* nodes I wanted to navigate, but I don’t think this is required.
8. To make the agents navigate, you need a bit of code. Set their target position like this:

```
navigation_agent.target_position = Vector2(24, 7)
```

9. They won't move by themselves, however. You need a bit of code in the `_physics_process` function to handle this, e.g.

```
if not navigation_agent.is_navigation_finished():
  var next_point = navigation_agent.get_next_path_position()
  var direction = (next_point - character.global_position).normalized()
  character.global_position += direction * speed * delta
```



Settings for the NavigationRegion2D

10. Setting the "Parsed Geometry Type" to "Meshes and Static Colliders" means that baking the polygon will take into account nodes with static colliders when they fall within the region. I've only been able to use this with the "Source Geometry Mode" set to "Group With Children", where I also specify a group name in the "Source Geometry Group Name"

field. For instance, I would assign the named group to *StaticBody2D* nodes in my scene.

Multiplayer Concepts: Peer-to-Peer (P2P) Model using Godot

Description

The **Peer-to-Peer (P2P) model** is a type of multiplayer network setup where **each player acts as both a client and a server**.

Instead of having one central authority, all peers (players) **communicate directly** and share game data such as player position, score, and actions.

Godot allows you to implement P2P networking using **ENet** or the **High-Level Multiplayer API**.

Each player can start a connection and exchange data directly with others over the network.

Uses

- Ideal for **small-scale multiplayer games** (2–4 players).
- Used in **LAN or local Wi-Fi games** where latency is low.
- Perfect for **co-op games**, **turn-based games**, or **simple multiplayer prototypes**.

  📌 *Example:* A 2-player car racing game or Pong clone where both players exchange position data directly.

Features of Peer-to-Peer Model

- Each peer is **equal** — there's no dedicated server.
- Every peer maintains a copy of the game state.

- Lower latency since messages don’t pass through a server.
- Works well for **local or small network setups**.

Advantages

✓ No server hosting required (reduces cost).
✓ Simple architecture for few players.
✓ Good for quick testing and prototyping.

Disadvantages

× Hard to maintain game state consistency.
× Cheating is easier (no server authority).
× NAT/firewall issues can make direct connections harder online.

Steps to Implement P2P Multiplayer in Godot

Step 1: Create a New Scene

Create a scene called P2PGame.tscn with a root node named Main.
Attach a script named p2p.gd.

Scene Hierarchy:

Main (Node)

Step 2: Setup ENet for Networking

Add code in p2p.gd:

```
extends Node


var peer


func _ready():
    print("P2P Multiplayer System Ready")
```

Step 3: Create and Connect Peers

In a P2P model, one player usually **hosts**, and the others **connect** to it.

To Create a Host:

```
func start_host():
    peer = ENetMultiplayerPeer.new()
    peer.create_server(1234, 4)  # port 1234, up to 4 players
    multiplayer.multiplayer_peer = peer
    print("P2P Host started on port 1234")
```

To Connect as a Peer:

```
func connect_to_peer(ip: String):
    peer = ENetMultiplayerPeer.new()
    peer.create_client(ip, 1234)
    multiplayer.multiplayer_peer = peer
    print("Connected to peer at ", ip)
```

Step 4: Handle Connection Events

Godot automatically provides signals to manage peer connections.

```
func _ready():
    multiplayer.peer_connected.connect(_on_peer_connected)
    multiplayer.peer_disconnected.connect(_on_peer_disconnected)


func _on_peer_connected(id):
    print("Peer connected: ", id)


func _on_peer_disconnected(id):
    print("Peer disconnected: ", id)
```

Step 5: Share Game Data (Using RPCs)

In a P2P game, all peers send updates to each other.
Example: Sharing player position.

```
@rpc(any_peer)
```

```
func update_position(pos: Vector2):
    $Player.position = pos


func _physics_process(delta):
    if multiplayer.is_server():
        rpc("update_position", $Player.position)
```

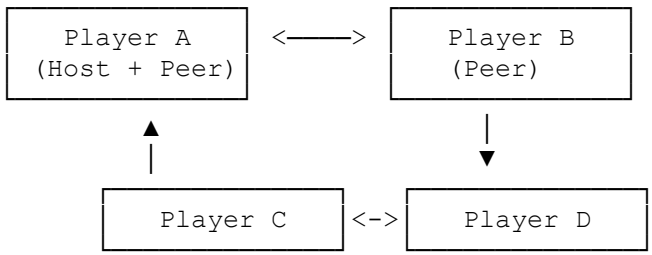
Each peer sends its position to others, keeping everyone synchronized.

Example Flow

1. Player A starts the host using start_host().
2. Player B connects using connect_to_peer("127.0.0.1").
3. Both share data using RPCs (e.g., position, score).
4. Every peer updates its game state in real-time.

5. Pictorial Representation

Peer-to-Peer Networking Diagram:



Multiplayer Concepts: Server-Based Model using Godot

Description

In a **Server-Based Multiplayer Model**, one computer (or device) acts as the **server**, and all other players connect as **clients**.
The **server is authoritative**, meaning it manages the true game state (positions, scores, physics, etc.) and sends updates to all clients.

This model ensures fairness, synchronization, and better control — making it the **standard for most online multiplayer games**.

Godot supports this model using:

- **ENet (Low-Level API)** — for custom networking.
- **High-Level Multiplayer API** — for simpler setup using RPCs (Remote Procedure Calls).

Uses

- Online multiplayer games that require **synchronization and fairness**.
- **Competitive games** like shooters, battle arenas, or racing.
- **Massively multiplayer (MMO)** or co-op games with many players.
- **LAN/Internet-based gameplay** with one authoritative server.

📌 *Example:* Games like **PUBG, Valorant, or Minecraft Servers** — where the server handles all player positions, attacks, and world updates.

Features of the Server-Based Model

- One **main server** maintains the actual game state.
- Clients only send **inputs or actions** to the server.
- Server sends **updated information** to all clients.
- **Server prevents cheating** by verifying all actions.
- **Stable synchronization** between multiple clients.

Advantages

✓ Centralized control (server decides final state).
✓ Secure — hard for clients to cheat.
✓ Works well for large multiplayer setups.
✓ Easier to maintain consistent gameplay.

Disadvantages

✗ Requires a host or dedicated server (extra cost).
✗ Slightly higher latency (data must go through the server).
✗ More setup complexity than P2P.

Steps to Implement Server-Based Multiplayer in Godot

Step 1: Create a Main Scene

Create a scene named ServerGame.tscn with a root Node.

Attach a script named server_game.gd.

Scene Hierarchy:

ServerGame (Node)

├── Player (KinematicBody2D)

└── Other Game Nodes…

Step 2: Setup Server and Client

Add the following to server_game.gd:

```
extends Node

var peer

func _ready():
    print("Server-Based Multiplayer Ready")

func start_server():
    peer = ENetMultiplayerPeer.new()
    peer.create_server(1234, 10) # Port 1234, max 10 clients
    multiplayer.multiplayer_peer = peer
    print("Server started on port 1234")

func connect_to_server(ip: String):
    peer = ENetMultiplayerPeer.new()
    peer.create_client(ip, 1234)
    multiplayer.multiplayer_peer = peer
    print("Connected to server at", ip)
```

Step 3: Handle Connection Events

Handle when clients connect or disconnect:

```
func _ready():
    multiplayer.peer_connected.connect(_on_peer_connected)
```

```
    multiplayer.peer_disconnected.connect(_on_peer_disconnected)


func _on_peer_connected(id):
    print("Client connected:", id)
    rpc_id(id, "welcome_message", "Welcome to the Server!")


func _on_peer_disconnected(id):
    print("Client disconnected:", id)
```

Step 4: Synchronize Game State Using RPC

Use **Remote Procedure Calls** to share information between server and clients.

Example: Broadcasting a player's position from the server:

```
@rpc("any_peer")
func update_position(pos: Vector2):
    $Player.position = pos
```

Server sends updates to all clients:

```
func _physics_process(delta):
    if multiplayer.is_server():
        rpc("update_position", $Player.position)
```

Clients send their input to server:

```
func send_input_to_server(pos: Vector2):
    rpc_id(1, "update_position", pos) # Send to server (ID 1)
```

Step 5: Manage Game Authority

- The **server** decides what is valid (movement, attacks, collisions).
- The **clients** only send input (like "move left" or "shoot").
- This ensures fairness and consistency.

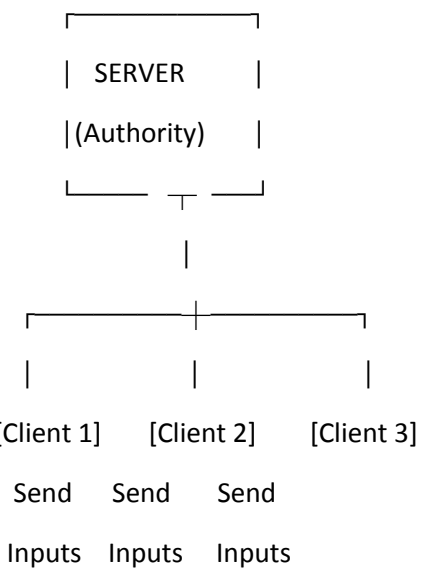
Example Flow

1. **Server** is started by Player A using start_server().
2. **Clients** connect using connect_to_server("Server_IP").

3. Server spawns players for each client.
4. Server constantly updates and syncs positions, scores, and actions using RPCs.
5. Clients display the updated game state in real time.

Pictorial Representation

Server-Based Multiplayer Network Flow:



Flow Explanation:

- Clients send actions to the server (Move, Jump, Shoot).
- Server validates and updates the world.
- Server broadcasts changes to all clients.

ENet (Low-Level API) in Multiplayer using Godot

Description

ENet is a **low-level networking library** that Godot uses for multiplayer communication.
It is built on top of **UDP (User Datagram Protocol)** and provides features like:

- Reliable delivery of packets (like TCP)
- Unreliable delivery (for faster performance)
- Connection management (connect, disconnect, timeout)

Godot uses ENet through the **ENetMultiplayerPeer** class.
This allows you to create both **servers** and **clients** for your multiplayer game.

ENet is best suited for **real-time games** where performance matters (e.g., shooting, racing, or action games).

Uses

- To create **custom multiplayer logic** (without high-level API).
- For **real-time gameplay** where low latency is important.
- To build both **server and client** directly using UDP.
- To understand the fundamentals of **how multiplayer works internally** in Godot.

📌 *Example:* You can build a racing game where every car's position is sent over ENet for instant updates.

Features of ENet

- **Reliable and Unreliable Packets:** You can choose which data must arrive (e.g., chat messages) and which can be skipped (e.g., positions).
- **Packet Channels:** You can send different kinds of data on different channels.
- **Peer Management:** ENet keeps track of all connected clients.
- **Low Latency:** Ideal for fast-paced games.

Steps to Implement ENet Multiplayer in Godot

Step 1: Create a New Scene

Create a main scene called NetworkDemo.tscn with a Node as the root.
Attach a new script called network.gd.

Scene Hierarchy:

NetworkDemo (Node)

Step 2: Setup Variables

Add the following code in network.gd:

```
extends Node


var peer


func _ready():
    print("Multiplayer ready")
```

Step 3: Create a Server

To create a server, use create_server():

```
func start_server():
    peer = ENetMultiplayerPeer.new()
    var result = peer.create_server(1234, 10)  # Port 1234, 10 max clients
    if result == OK:
        multiplayer.multiplayer_peer = peer
        print("Server started on port 1234")
    else:
        print("Failed to start server")
```

Step 4: Connect as a Client

To connect to the server:

```
func connect_to_server(ip: String):
    peer = ENetMultiplayerPeer.new()
    var result = peer.create_client(ip, 1234)
    if result == OK:
        multiplayer.multiplayer_peer = peer
        print("Connected to server")
    else:
        print("Connection failed")
```

Step 5: Handle Connection Signals

You can handle connection events using signals:

```
func _on_peer_connected(id):
    print("Client connected: ", id)

func _on_peer_disconnected(id):
    print("Client disconnected: ", id)

func _on_connected_to_server():
    print("Connected to server!")
```

```
func _on_connection_failed():
    print("Failed to connect to server!")

func _on_server_disconnected():
    print("Disconnected from server!")
```

Register these signals in _ready():

```
func _ready():
    multiplayer.peer_connected.connect(_on_peer_connected)
    multiplayer.peer_disconnected.connect(_on_peer_disconnected)
    multiplayer.connected_to_server.connect(_on_connected_to_server)
    multiplayer.connection_failed.connect(_on_connection_failed)
    multiplayer.server_disconnected.connect(_on_server_disconnected)
```

Step 6: Send and Receive Messages

Use **Remote Procedure Calls (RPCs)** to send messages between peers:

```
@rpc(any_peer)
func send_message(msg: String):
    print("Received message: ", msg)
```

Client can call:

```
@rpc
func send_message_to_server(msg: String):
    rpc_id(1, "send_message", msg)  # Send to server (ID 1)
```

Server can broadcast:

```
@rpc
func broadcast_message(msg: String):
    rpc("send_message", msg)
```

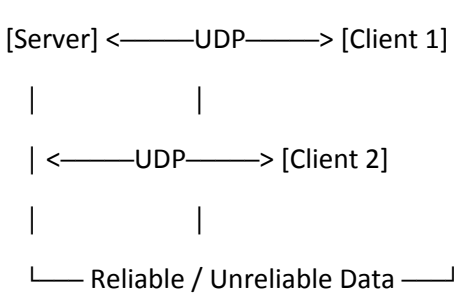
Example Flow

1. One player runs the game in **Server Mode** (start_server()).
2. Other players run the game in **Client Mode** (connect_to_server("127.0.0.1")).

3. They exchange messages or gameplay data using **RPCs**.

Pictorial Representation

ENet Networking Flow:



Godot High-Level Multiplayer API

Description

The **High-Level Multiplayer API** in Godot provides an easier way to implement multiplayer features without manually handling low-level networking code (like in ENet).
It is built **on top of ENet**, so it still uses **UDP** underneath, but it automatically manages:

- Connections (server and clients)
- Player synchronization
- Remote Procedure Calls (RPCs)
- Ownership and authority of nodes

This API allows developers to create multiplayer games with **very little code** — making it ideal for both beginners and professionals.

Uses

- For **multiplayer synchronization** of game objects (like players, bullets, or scores).
- For **real-time games** (shooters, racing, platformers).
- For **turn-based games** (chess, cards).
- To quickly create **LAN or online games** using the built-in functions.

📌 *Example:* You can make a 2-player shooting game where both players' positions and bullets sync automatically using RPC calls.

Features

- **Easy Setup:** Only a few lines of code to create a server or client.

- **Built-in Synchronization:** Automatically syncs node states and functions.
- **Remote Procedure Calls (RPCs):** Allows you to call a function on remote peers easily.
- **Authority Control:** The server can control what clients can or cannot do.
- **Works with ENet:** Uses ENetMultiplayerPeer internally for fast, reliable networking.

Steps to Use Godot's High-Level Multiplayer API

Step 1: Create a New Scene

Create a new scene named **MultiplayerDemo.tscn**
Add a root Node and attach a script named multiplayer_demo.gd.

Scene Hierarchy:

MultiplayerDemo (Node)

Step 2: Setup Server or Client

In multiplayer_demo.gd, write:

```
extends Node

var peer

func _ready():
    print("Multiplayer system ready")

func create_server():
    peer = ENetMultiplayerPeer.new()
    peer.create_server(1234, 10) # Port 1234, max 10 clients
    multiplayer.multiplayer_peer = peer
    print("Server started")

func connect_to_server(ip: String):
    peer = ENetMultiplayerPeer.new()
    peer.create_client(ip, 1234)
```

```
multiplayer.multiplayer_peer = peer
print("Connected to server")
```

Step 3: Add RPC Function

RPCs (Remote Procedure Calls) allow you to call functions across the network.

```
@rpc
func announce(msg: String):
    print("Received:", msg)
```

Now, from the client, you can send messages to all peers:

```
func send_message_to_all():
    rpc("announce", "Hello from " + str(multiplayer.get_unique_id()))
```

Step 4: Handle Player Spawning

You can use **RPCs** to spawn player nodes across all peers:

```
@rpc("any_peer", "call_local")
func spawn_player(id):
    var player = preload("res://Player.tscn").instantiate()
    player.name = str(id)
    add_child(player)
    print("Spawned player:", id)
```

The server can then spawn players when clients connect:

```
func _on_peer_connected(id):
    rpc("spawn_player", id)
```

Step 5: Manage Connection Signals

Handle connection events:

```
func _ready():
    multiplayer.peer_connected.connect(_on_peer_connected)
    multiplayer.peer_disconnected.connect(_on_peer_disconnected)


func _on_peer_connected(id):
```

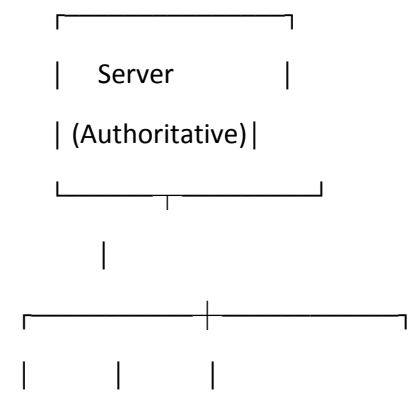
```
    print("Client connected:", id)


func _on_peer_disconnected(id):
    print("Client disconnected:", id)
```

Example Flow

1. Player A starts the game as a **server**.
2. Player B connects using the **client** function.
3. The server uses **RPCs** to broadcast player data (positions, health, messages).
4. All players see synchronized movement and game state.

Pictorial Representation

Network Model (High-Level API):



Legend:

- Server sends updates to all clients.
- Clients send RPC calls back to the server (e.g., move, shoot, chat).